

Indian Institute of Technology Jodhpur

PG Diploma in Artificial Intelligence

Natural Language Understanding

Assignment 2

Sanskrit-to-English Neural Machine Translation using Attention-based Sequence-to-Sequence Model

Submitted by

Monika Sharma

Roll No.: G25AIT1094

Abstract:

This project is based on a custom implementation of a Neural Machine Translation (NMT) system in python torch with attention-based Sequence-to-Sequence architecture for translating Sanskrit to English. The model is composed of a bidirectional LSTM Encoder, Bahdanau attention mechanism and LSTM Decoder. During pre-processing, word-level tokenization, construction of the vocabulary, numerical encoding and padding were performed. The model was trained with the Adam optimizer with teacher forcing. To improve generalization, label smoothing, learning-rate scheduling and early stopping were incorporated. The final model achieved a BLEU score of 2.419 and a BERTScore (F1) of 0.817 with an inference time of ~ 0.07 seconds and 15.85 million trainable parameters.

1. Introduction

Neural Machine Translation trains the map between source and target languages directly via deep-learning models. Sanskrit is a morphologically complex and difficult language, and has limited parallel corpora compared to other languages, which makes automatic translation difficult. The aim of this assignment was to develop a full-fledged Sanskrit to English translation system without any external Sanskrit to English or English to Sanskrit translation APIs or pre-trained translation models.

2. Dataset and Preprocessing

The given corpus consists of pairs of sentences in Sanskrit and English. The training, development and test sets contain 10,000, 1,000 and 1,000 sentence pairs respectively. The common Source id was used to merge the source and target files. The preprocess steps included whitespace tokenization, constructing vocabulary (with adding special tokens <PAD>, <SOS>, <EOS> and <UNK>), encoding and adding sequence padding to the numbers.

Dataset	Sentence Pairs
Training	10000
Development	1000
Test	1000

3. Proposed Architecture

The architecture is proposed based on Encoder-Decoder paradigm. The proposed encoder is a two-layer Bidirectional LSTM to generate contextual representations of Sanskrit sentences. Bahdanau attention calculates alignment scores between the encoder output and decoder hidden state, enabling decoder to attend to the most relevant source words in each decoding step. The decoder is an LSTM followed by a fully connected layer to predict the next English token.

Architecture Pipeline

Sanskrit Sentence → Word Embedding → BiLSTM Encoder → Bahdanau Attention → Context Vector → LSTM Decoder → Fully connected Linear Layer + Softmax → English Translation

4. Training Procedure

Adam optimizer with a learning rate of 0.001 was used to train the model. This was done using a cross-entropy loss with label smoothing (0.1). The ratio of teacher forcing was set at 0.5. To prevent overfitting and enhance convergence, gradient clipping, ReduceLROnPlateau learning-rate scheduling and early stopping were added.

Hyperparameter	Value
Embedding Dimension	256
Hidden Dimension	256
Encoder Layers	2
Decoder Layers	1
Batch Size	64
Epochs	15
Optimizer	Adam
Learning Rate	0.001
Teacher Forcing	0.5
Regularization	Label Smoothing, LR Scheduler, Early Stopping



Figure 1. Training and Validation Loss Curves. The validation loss decreases initially and starts increasing after approximately Epoch 7, indicating overfitting. Label smoothing, learning-rate scheduling and early stopping improved the model's generalization.

Regularization Techniques: In the first experiments, it was noted that the training loss was continuously decreasing but the validation loss began to increase after some epochs. This behavior meant that the model starts to start overfitting the training data. Overfitting is when the model becomes too close to the training data, and performs poorly on new data. Several regularization techniques were added to the learning process to improve generalisation and achieve better translation performance.

Label Smoothing : The small probability was used for the incorrect target classes instead of a one-hot distribution and was used as regularization; this is known as label smoothing. This helps the model not overconfident in its predictions and to generalize well. For this implementation the value of 0.1 was set for label smoothing and the Cross-Entropy loss function was chosen.

Learning Rate Scheduler : A ReduceLROnPlateau learning rate scheduler was used to automatically decrease the learning rate when the validation loss was no longer improving. Instead of training with a relatively high learning rate, the scheduler slowly decreased the learning rate, letting the optimizer do smaller updates on the parameters which allowed the optimizer to find a better local optimum.

Early Stopping : The model was originally trained for 15 epochs, but after a few epochs, the loss in the validation set stopped getting better so these were dropped. To this end, Early Stopping was employed to automatically stop the training procedure if the validation loss did not decrease for a given number of successive epochs. This avoided overfitting, unnecessary training and ensured the best model on the validation set was preserved.

The reasons for using these techniques : The loss curves for training and validation are shown in Figure 1. The training loss gradually drops during the training process, but the validation loss starts to rise after about Epoch 7, showing that the model is no longer learning the translation patterns and is instead memorizing the training data. Label smoothing and learning rate scheduling and early stopping were added to boost the model generalization, and the BLEU score was increased from 1.863 to 2.419 with a high semantic similarity of BERTScore 0.817.

5. Experimental Results

Metric	Baseline	Improved
BLEU	1.863	2.419
BERTScore (F1)	0.817	0.817
Inference Time	0.07 s	0.07 s
Parameters	15.85 Million	15.85 Million

The better model improved BLEU score by around 30% over the baseline with high semantic similarity score as BERTScore. Also, training and validation curves revealed that the validation loss began to increase after some epochs, which encouraged the use of early stopping and learning-rate scheduling.

6. Translation Examples and Error Analysis

Source	Reference	Prediction	Observation
अधुना, अस्माकं प्रोग्राम् पश्यामः।	Now let us move on to our program.	Now, let us see our program.	Semantic meaning preserved.
अहं तमिल्-भाषायां वदामि।	I will speak in Tamil.	I will give the <UNK>.	Rare word translated incorrectly.
युवाम् अस्माकं हृदयानि अहरतम्।	You both stole our hearts.	Our our our <UNK>.	Repetition and UNK generation.
अधुना संसूचके टङ्कयतु - "chmod space 751 space example1" Enter नुदतु	Now type the command: chmod space 751 space example1 press Enter.	Now, type: ls space <UNK> space <UNK> space <UNK>.	Partially Correct. The model correctly recognizes the instructional context ("Now, type"), but fails to reproduce rare Linux commands due to limited vocabulary coverage.
एषः भवतां शीतली प्राणायामस्य एकः क्रमः सम्पूर्णः अभवत् ।	This completes one round of Sitali Pranayama.	It will also the <UNK> <UNK> in the <UNK>.	Incorrect Translation. The model struggles with domain-specific terminology ("Sitali Pranayama") and produces several unknown tokens, indicating poor representation of rare vocabulary.

7. Discussion

The attention mechanism enhanced the alignment between source and target sequences and allowed for meaningful translations in a number of short and medium length sentences. But, the long Sanskrit sentences and uncommon words and technical terms often resulted in the creation of <UNK> tokens. The relatively poor BLEU is mainly attributable to the small training corpus and word-level tokenization, while the relatively high BERTScore shows that there is usually a conservation of semantic information.

8. Conclusion

An attention based Seq2Seq architecture was successfully implemented from scratch resulting in a complete Sanskrit to English neural machine translation system. Using training enhancements like label smoothing, learning-rate scheduling and early stopping, BLEU was improved to 2.419. The next steps are beam search decoding, subword tokenization and Transformer-based multilingual translation models.

References

- Bahdanau D. et al. Neural Machine Translation by Jointly Learning to Align and Translate, 2015.
- Sutskever I. et al. Sequence to Sequence Learning with Neural Networks, 2014.
- Papineni K. et al. BLEU: A Method for Automatic Evaluation of Machine Translation, 2002.
- Zhang T. et al. BERTScore: Evaluating Text Generation with BERT, 2020.
- PyTorch Documentation.